Parametrização e resolução do problema dos missionários e canibais

Jaime Antonio Daniel Filho[1], **Diego Ribeiro Chaves**[1]

[1]Curso de Ciência da Computação, Universidade Federal de Santa Maria (UFSM)

jafilho@inf.ufsm.br, drchaves@inf.ufsm.br

1. Introdução

O problema dos missionários e canibais é um clássico da inteligência artificial e da teoria da busca. Na formulação original, três missionários e três canibais precisam atravessar um rio usando um barco com capacidade para duas pessoas, respeitando a restrição de que os canibais nunca podem superar em número os missionários em nenhum dos lados da margem, caso contrário, os missionários seriam devorados.

Neste trabalho, generalizamos o problema ao parametrizar o número de pares missionário-canibal como N e a capacidade do barco como K. O objetivo é estudar como essas duas variáveis afetam a complexidade da busca: o número de estados explorados, o consumo de memória, o tempo de execução e a profundidade da solução ótima. A busca em largura (BFS) é utilizada por garantir que a primeira solução encontrada possui o menor número de movimentos possível.

2. Metodologia

Nesta seção, detalharemos a abordagem adotada para a modelagem, resolução e coleta de métricas do problema dos missionários e canibais.

2.1. Formulação do problema

O problema foi formalizado utilizando estruturas de dados para representar os estados, que indicam a quantidade de canibais e missionários em cada margem e a posição do barco, em que, se um estado for considerado válido, é colocado em uma fila para ser explorado. A fila é explorada utilizando a abordagem FIFO, o que caracteriza o algoritmo de busca em largura (BFS).

2.1.1. Estado inicial

O espaço de estados é definido pela tripla (L, C, M), onde:

- $L \in \{0, 1\}$ indica em qual margem o barco está (0 = margem de origem, 1 margem de destino);
- $C \in \{0, ..., N\}$ é o número de canibais na margem L;
- $M \in \{0, ..., N\}$ é o número de missionários na margem L.

O estado inicial é $(0, N, N)$, ou seja, todos os N missionários e N canibais estão na margem de origem, juntamente com o barco. Por outro lado, o estado final é $(1, N, N)$, ou seja, todos estão na margem de destino.

2.1.2. Função sucessora

A cada passo, o barco transporta entre 1 e K pessoas da margem onde ele se encontra para a outra. Os pares (m, c) de missionários e canibais embarcados devem satisfazer:

- $m + c \geq 1$ e $m + c \leq K$;
- Em ambas as margens, após a travessia, o número de missionários deve ser maior ou igual ao de canibais, a menos que não haja missionário algum naquela margem;
- Adicionalmente, adotamos a restrição de que dentro do próprio barco o número de missionários também deve ser maior ou igual ao de canibais, exceto quando não há missionários a bordo.

Como a função sucessora pode transicionar para um estado que já foi visto anteriormente, cada estado visto é armazenado em uma tabela *hash*, comportamento que batizamos de deduplicação. Desse modo, um estado sucessor somente é adicionado à fila de avaliação, caso não for encontrado na tabela *hash*. Isso impede que a BFS revisite os mesmos estados indefinidamente. A deduplicação é controlada pelo parâmetro D (0 = desativado, 1 = ativado).

O caminho, isto é, as transições que foram realizadas para alcançar a solução é implementado por meio do *backtracking*. A cada estado adicionado à fila, armazena-se o seu estado pai em outra tabela *hash*. Ao final da execução, percorre-se a tabela de pais de trás para frente, buscando o pai do estado final $(1, N, N)$ até chegarmos no estado que possui o estado inicial $(0, N, N)$ como pai.

2.2. Configurações testadas e métricas

Os experimentos foram conduzidos variando:

- K de 1 a 128 em incrementos de 1;
- N de 1 a 256 em incrementos de 1, e em seguida em progressão geométrica de base 2 até o consumo de memória exceder 4GB.

Para monitorar o desempenho e a evolução da busca, registrou-se um conjunto de métricas a cada nível de profundidade percorrido na árvore de estados. Essa abordagem permite observar o crescimento da fila e da tabela *hash* de forma dinâmica, em vez de avaliar apenas o estado final.

A Tabela 1 detalha os dados capturados em cada ponto de amostragem. A memória é estimada conforme a carga na estrutura da fila e na estrutura da tabela *hash*.

Tabela 1. Métricas registradas por nível de profundidade.

Métrica	**Descrição**
`depth`	Profundidade atual da árvore de busca.
`queue size`	Número de estados na fila a serem avaliados.
`table size`	Número de estados armazenados pela tabela *hash*.
`table buckets`	Número de baldes alocados pela tabela *hash*
`states explored`	Número cumulativo de estados avaliados.
`states skipped`	Número de estados ignorados por já terem sido vistos.
`memory usage`	Estimativa do consumo de memória em bytes.
`time elapsed`	Tempo decorrido em nanosegundos.

Abaixo é apresentado um formato de amostra da saída padronizada. O formato de resultado inicia sempre com os parâmetros utilizados e finaliza com o número de travessias mínimas para a solução.

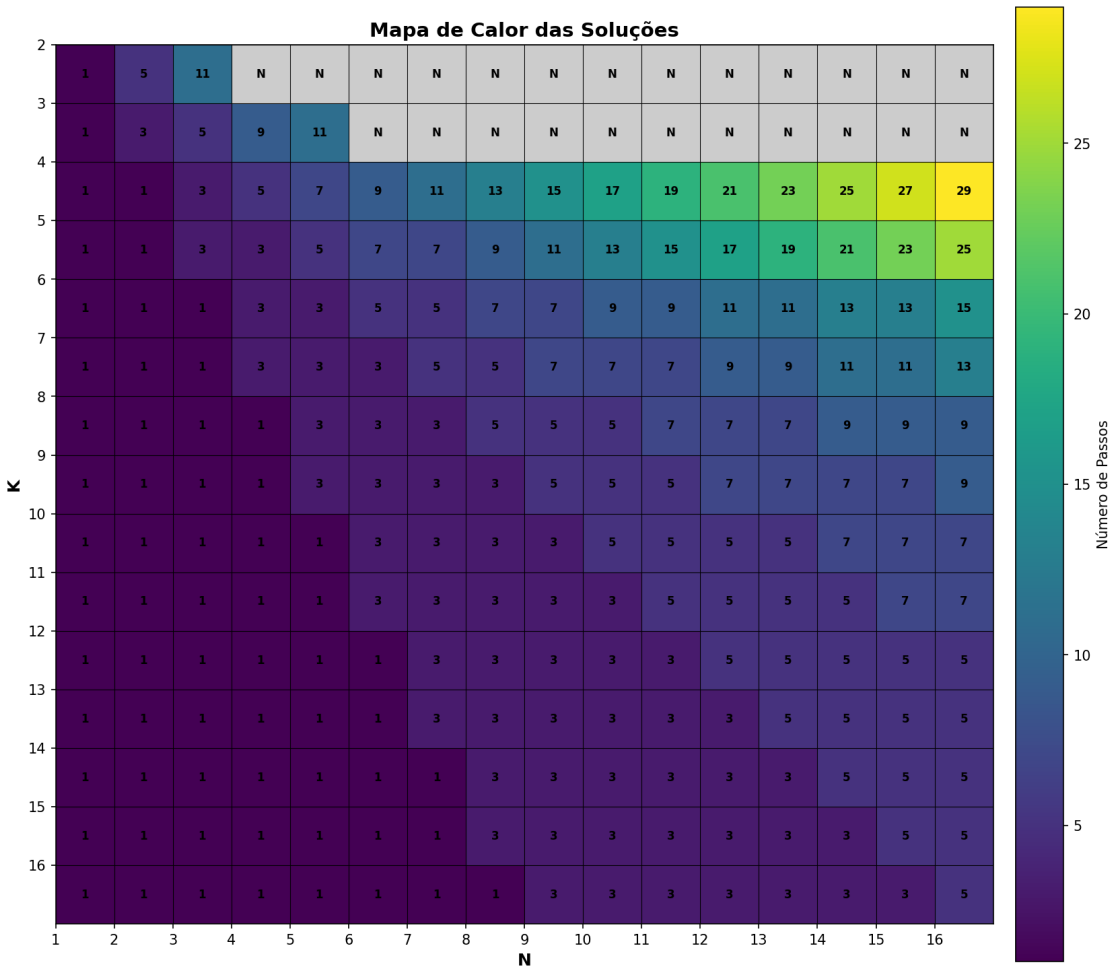
```
n: 128, k: 64, deduplicate: 1, memory limit: 4294967296

depth: 1, queue size: 95, table size: 97, table buckets: 127, states explored: 1, states skipped: 0, memory usage: 10566, time elapsed: 32940
depth: 2, queue size: 93, table size: 191, table buckets: 257, states explored: 97, states skipped: 3042, memory usage: 16410, time elapsed: 56101
depth: 3, queue size: 93, table size: 285, table buckets: 541, states explored: 191, states skipped: 9027, memory usage: 26046, time elapsed: 232473
depth: 4, queue size: 93, table size: 379, table buckets: 541, states explored: 285, states skipped: 14484, memory usage: 28866, time elapsed: 304448
depth: 5, queue size: 31, table size: 411, table buckets: 541, states explored: 379, states skipped: 17524, memory usage: 26850, time elapsed: 449746
depth: 6, queue size: 31, table size: 443, table buckets: 541, states explored: 411, states skipped: 18549, memory usage: 27810, time elapsed: 505285
depth: 7, queue size: 89, table size: 533, table buckets: 541, states explored: 443, states skipped: 19917, memory usage: 33294, time elapsed: 558032
depth: 8, queue size: 152, table size: 686, table buckets: 1109, states explored: 533, states skipped: 25432, memory usage: 54540, time elapsed: 728677
depth: 9, queue size: 70, table size: 757, table buckets: 1109, states explored: 686, states skipped: 32687, memory usage: 52734, time elapsed: 767541
depth: 9, queue size: 8, table size: 765, table buckets: 1109, states explored: 757, states skipped: 35235, memory usage: 49998, time elapsed: 861491

solution: 9
```

3. Resultados

Nesta seção, analisaremos os resultados obtidos com os experimentos variando os parâmetros K e N limitados por 4GB de memória. No total, foram feitos 43068 testes, que produziram mais de 286MB de dados.

A Figura 1 mostra um mapa de calor das soluções para alguns cenários de K e N. Pode-se observar que para $K = 2$, existem poucas soluções ($N \in \{1, 2, 3\}$), para $K = 3$ existem mais alguns cenários com solução ($N \in \{1, 2, 3, 4, 5\}$). Com valores de K grandes e N pequenos, as soluções exigem um menor de número de travessias.

Figura 1. Mapa de calor do número de travessias

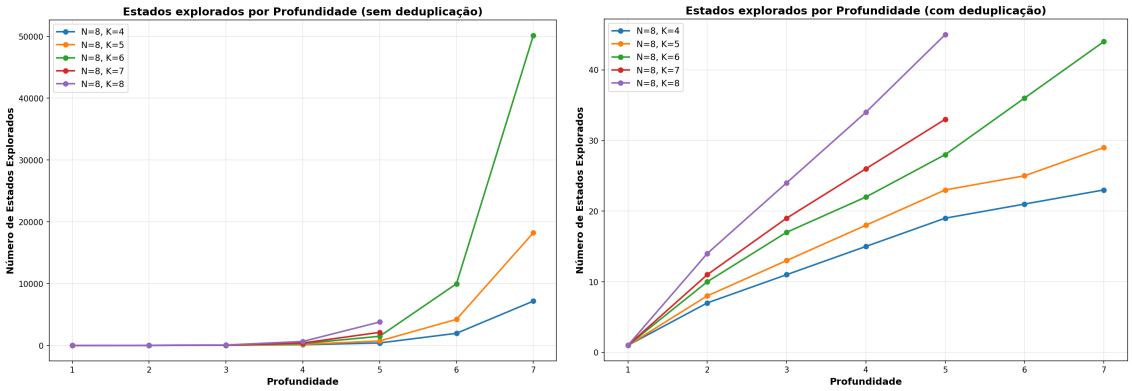


3.1. Importância da deduplicação

Para ilustrar o impacto da deduplicação, fixamos $N = 8$ e comparamos as execuções com $K \in \{4, 5, 6, 7, 8\}$, plotando o número acumulado de estados explorados no eixo Y conforme a profundidade avança no eixo X. A Figura 2 apresenta esse resultado, apresentando a execução sem deduplicação no painel esquerdo e a execução com deduplicação no painel direito.

A hipótese central é que barcos menores produzem soluções mais profundas, mais idas e vindas são necessárias para atravessar todos, enquanto barcos maiores produzem soluções rasas, mas com maior fator de ramificação em cada nível. Sem deduplicação, os estados repetidos se acumulam exponencialmente em ambos os casos, tornando a busca intratável mesmo para N moderados. Com deduplicação, o número de estados únicos é limitado pela cardinalidade do espaço de estados.

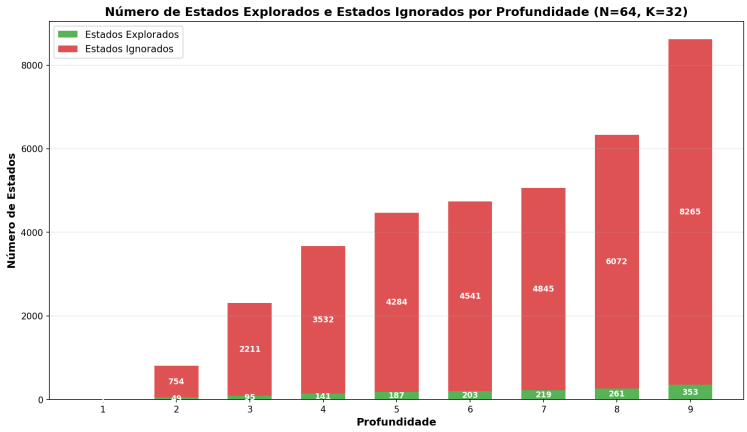
Figura 2. Comparação do número de estados explorados em função da profundidade da busca, com e sem deduplicação.



Sem deduplicação, o número de estados cresce de forma acentuada a cada nível da árvore, pois os mesmos estados são reenfileirados repetidamente através de caminhos distintos. Com deduplicação, as curvas crescem de forma muito mais contida e convergem rapidamente, refletindo o fato de que o espaço de estados únicos é finito. Esse contraste justifica a adoção obrigatória da tabela *hash* em qualquer execução prática, uma vez que, além de reduzir o consumo de memória e tempo, ela é a única garantia de terminação nos casos sem solução.

A Figura 3 mostra a proporção de estados explorados e ignorados em função da profundidade para $N = 64$ e $K = 32$. Pode-se observar que a deduplicação é bem eficiente, pois faz com que o algoritmo não explore desnecessariamente estados já vistos.

Figura 3. Proporção de estados explorados e ignorados em função da profundidade

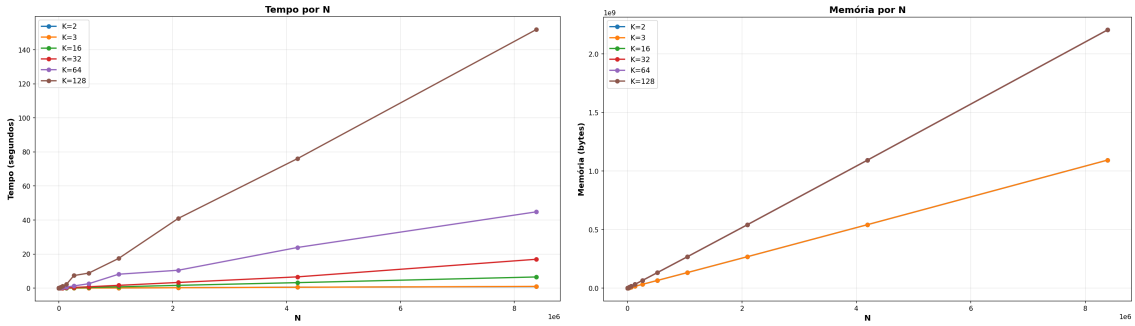


3.2. Desempenho em função de N

Para avaliar como a complexidade escala com o tamanho do problema, fixamos alguns valores representativos de K e variamos N no eixo X, plotando o tempo de execução e a memória de pico no eixo Y. A memória de pico é extraída do maior valor de *memory usage* reportado durante a execução.

Como a Figura 4 mostra, há um crescimento aparentemente linear da memória à medida que o N aumenta para um mesmo K, e um crescimento aparentemente linear do tempo.

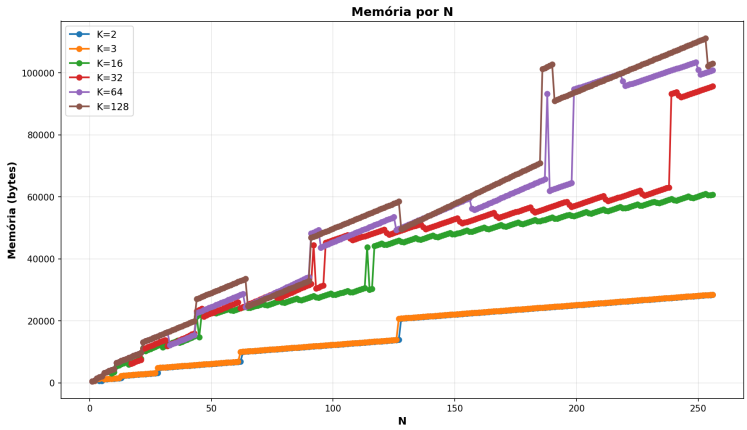
Figura 4. Complexidade de tempo e de memória em função de N



Observa-se que conforme esperado, tanto a memória, quanto o tempo de execução crescem à medida que o N também cresce para um mesmo K. Isso mostra que é necessário mais viagens de barco para resolver o problema, uma vez que temos mais passageiros.

A Figura 5 mostra com mais detalhes, para N limitado entre 1 e 256, esse comportamento da memória conforme o N aumenta.

Figura 5. Complexidade de memória em função de N limitado entre 1 e 256



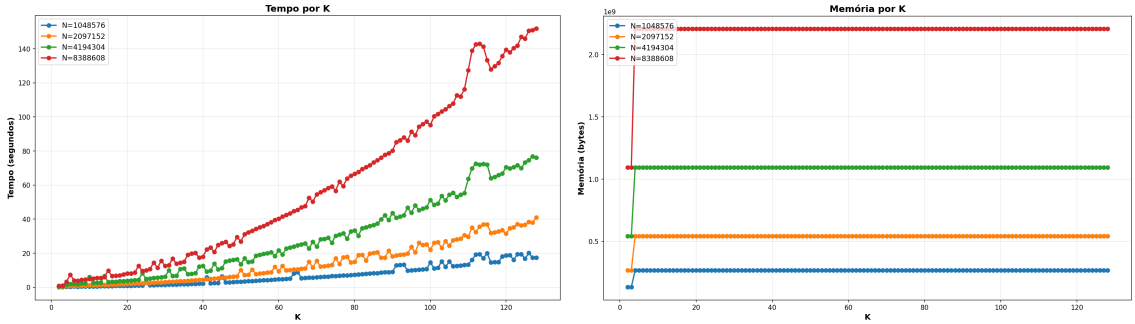
3.3. Desempenho em função de K

Para estudar o efeito isolado da capacidade do barco, fixamos valores representativos de N e variamos K no eixo X.

O comportamento do tempo de execução mostra um crescimento aparentemente exponencial no tempo de execução para um mesmo N à medida que o K aumenta, ou seja, quanto maior K, maior a ramificação da árvore, e precisa-se de mais tempo para percorrer sua largura, como pode ser visto na esquerda da Figura 6.

Nota-se que o pico da memória depende apenas do valor de N a partir de certo valor, mas que inicialmente, com valores de K pequenos, cresce rapidamente á medida que o K aumenta, depois mantendo a constância aparente na direita da Figura 6.

Figura 6. Complexidade de tempo e memória em função de K



4. Conclusão

Esse trabalho teve como objetivo generalizar o problema clássico dos canibais e missionários parametrizando o tamanho do barco e a quantidade de personagens. A abordagem com BFS se mostrou eficiente para encontrar a soluções ótimas com o mínimo de travessias. Os principais achados podem ser sintetizados por:

- A deduplicação via tabela *hash* se mostrou indispensável, sem ela o algoritmo se torna impraticável para N moderados e não há garantia de terminação em casos sem solução.
- O tempo e a memória crescem aparentemente de forma linear com N para K fixo.
- O aumento de K tem impacto mais agressivo no tempo acarretando em um crescimento aparentemente exponencial, pois amplia o fator de ramificação da árvore de busca.
- A memória satura a partir de certo K, passando a depender predominantemente de N, o que sugere que o espaço de estados únicos é limitado pela cardinalidade do espaço e não pela ramificação.